

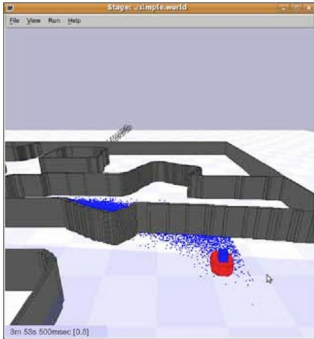


Figure 9: Plume Sim, the plume of gas is seen in blue

All quality simulation and control interfaces employ the client-server architecture; Player Project did that, and ROS has employed much the same. However, the command-line codes in ROS very closely resemble Bash commands. Here are some samples:

ROS commands	Bash equivalents
roscd	cd
roscat	cat
rosclear	clear
roscpp	cat
roslaunch	cat
rosls	ls

Table 2: ROS and Bash commands

ROS uses nodes that are connected to a master, *roscore*, which are the clients and server respectively.

Tools for simulation: *stage* and *gazebo*

Most of you reading this article will not have access to real robots, and hence simulation will be a nice way to get to know about ROS. Two very popular simulators for 2D and 3D simulations are *stage* and *gazebo*, respectively. Both these were part of the Player Project, and were forked and integrated into ROS. Once ROS electric is installed and working on your computer, *stage* can be installed using the following command:

rosmake stage

ROS uses a wrapper, *stageros*, which incorporates *stage* into ROS nodes. The simulations can be run using *roscore* (the master) and, in a different terminal window, you can start the *stage* simulations as follows:

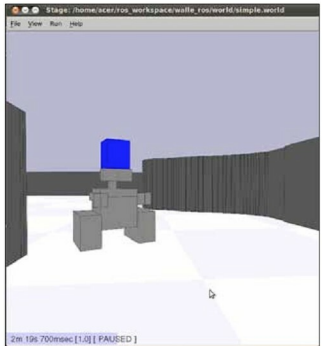


Figure 10: Wall-e like model in stage

```
roslaunch stage stageros `rospack find stage`/world/willow-erratic.world
```

That should pop up the screen shown in Figure 8. For a perspective view, click the *stage* window and press ‘R’, and you should be able to see a box-like erratic robot. More details on *stage* may be found at <http://www.ros.org/wiki/stage>. *Stage* is also invaluable in swarm robotics and multi-robot simulations.

An interesting example using *stage* is *plumesim* (Figure 9) which can simulate a plume of gas. More details on *plumesim* can be found at <http://www.ros.org/wiki/plumesim/Tutorials>. Another recreation is wall-e like model in *stage* (Figure 10).

Gazebo can be installed in much the same manner, as shown below:

```
rosmake gazebo worlds
```

A trial run with an empty world can be done with the following code:

```
roslaunch gazebo_worlds empty_world.launch
```

Gazebo provides 3D simulation, and it is more work to design a simulation with it, as compared to working with *stage*. Also, *gazebo* demands quality graphical memory, and makes heavy demands on systems resources. More details on *gazebo* can be found at <http://ros.org/wiki/gazebo> and <http://gazebo.sim.org/>.

Robot models made in Blender or Google SketchUp can easily be imported into *gazebo*. As an example, have a look at Jeff's repository, which has a *gazebo* model (Figure 12)